

Kandidatnummer:

2010, 2169, 2001, 2070, 2286, 2334, 2299, 2146

PRO202 - Smidig Prosjekt
Project Result, Group Porto



Denne besvarelsen er gjennomført som en del av utdannelsen ved Høyskolen Kristiania.
Høyskolen er ikke ansvarlig for oppgavens metoder, resultater, konklusjoner eller anbefalinger.

Table of contents

1 Introduction	3
1.1 Description of our product	3
1.2 Concept.....	3
1.3 Goals.....	4
2 How U in MNCH works	4
2.1 How to use the application	4
2.1.1 Login and registration	4
2.1.2 Select game	5
2.1.3 The infinite tasks.....	5
2.1.4 Endstory.....	8
2.1.5 Rewards.....	8
2.2 Setup development environment.....	9
2.3 How to test the application	16
3 Technical choices.....	18
3.1 Programming language	18
3.2 Database.....	18
3.3 Login system.....	19
3.4 Functionalities.....	22
3.5 Improvements	31
4 Theoretical choices	32
4.1 Design	32
4.1.1 Icons.....	32
4.1.2 Color choices	33
4.1.3 Fonts	33
4.2 UI Choices	34
4.3 UX Choices	34
4.4 Future UX Solutions.....	34
5 Prototype	35
5.1 Design tools	35
5.2 Prototypes.....	35
5.2.1 Prototyping.....	35
5.2.2 Wireframe	36
5.2.3 First prototype	36
5.2.4 Final prototype	37
6. Usertesting	39
6.1 Purpose of usertesting.....	39
6.2 Execution	39
6.3 Results	40
6.4 Improvements	40
6.5 Reflection	41
7 Further development.....	42

7.1 What would we further develop	42
7.2 Security reflections	43
8 Result evaluation	46
8.1 What have we learned	46
8.2 What are we satisfied with	47
8.3 What would we do differently	47
9 Sources	48
10 Attachments	49

1 Introduction

1.1 Description of our product

The solution we have created is based on one of the cases given by MUNCH. We were presented with two different cases from product owner, and we chose the following case:

“How can we make young adults (18-25) more engaged in Munch, through innovative use of new technologies, forms of experience and interaction that argue for the physical art experience + How can we measure and visualize the effect of the engagement we create through innovative use of technology?”

U in MNCH is the solution we made, which is an application built as a game, to inspire and engage young adults to become interested in Munch’s art. The game consists of clues and tasks to solve a mystery, which is based on a true story about what happened to some of Munch's paintings during WW2. (Alberge, 2023) The goal of the game is to find out what happened to a missing painting, and the true story will be revealed when the last clue is solved. The user will get small pieces of information about the story along the way from Munchgyver. He is the detective who has been working on the case, and needs help from the user to solve the tasks by connecting the clues. By looking at the progress of the user, and how many of the users that complete the mystery, we can track the effect of the engagement we are creating.

1.2 Concept

U in MNCH is made to make the MUNCH experience interactive and engaging for our target audience, by making the user engage in the art and physical space. We wanted to create a game that was somehow similar to the tasks you get in an escape room.

To solve the tasks and find clues for the mystery, the user is supposed to be in an exhibition at MUNCH. Solving a task will lead the user to the next clue, and the answers will be in the painting or in the physical space at MUNCH. The end goal of the game is to solve the mystery. When solved, the user will get the true story, which the mystery is based on. The

user will also get a reward, such as a free coffee or a ticket to an event, when finished with all the tasks.

1.3 Goals

The overall goal with the application is to create an engaging experience, by using technology to explore art in a new way. We wanted to create a solution that combines curiosity, art and technology. The game combines art with experiences, and is presented in a way that we hope captures the attention and curiosity of the user.

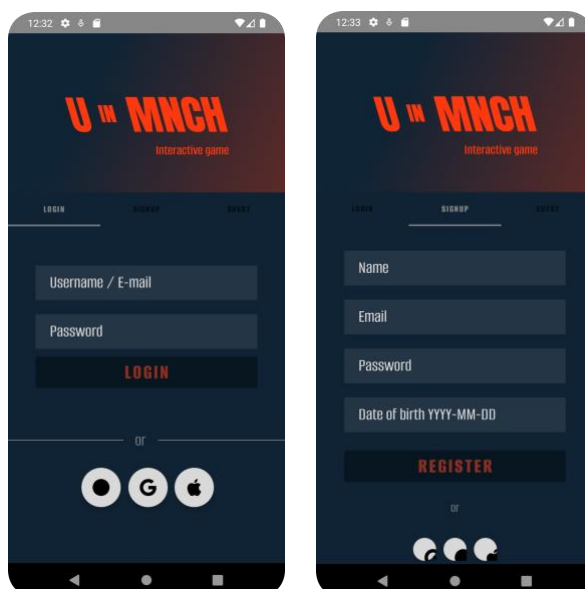
2 How U in MNCH works

You can find our source code here: https://github.com/ChrisGloom1/porto_u_in_mnch

To access the U in MNCH prototype, press this link: [Link to Prototype in Figma](#)

2.1 How to use the application

2.1.1 Login and registration



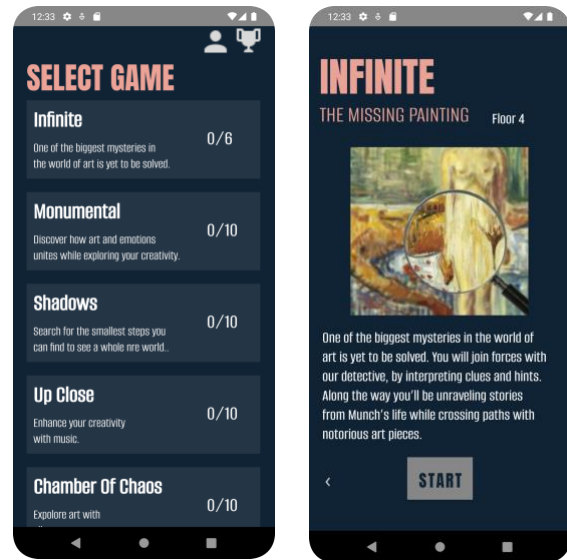
When the user opens the application for the first time, they will have to register an account. By clicking “register” they will get to the exhibitions.

2.1.2 Select game

After registration, the user will be presented with the different games.

The games are based on the exhibition names. By clicking on the name of the game they will get to the next page.

When pressing INFINITE, the next page will display information about what kind of game they can expect. They will get to know about a mystery and that they have to interpret clues to solve it.



2.1.3 The infinite tasks

When doing the tasks, the user will have an overview of what tasks they have achieved, and which tasks are missing.

The site is a scrolling site. To continue solving tasks, they click on the task they want to do. The name of the tasks is connected to what the user is supposed to do in the task.

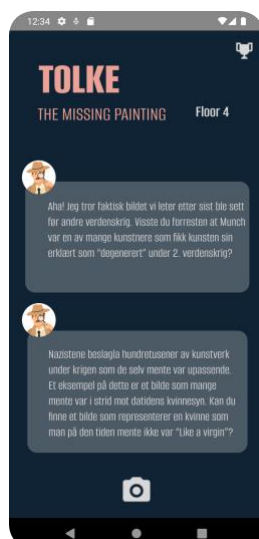
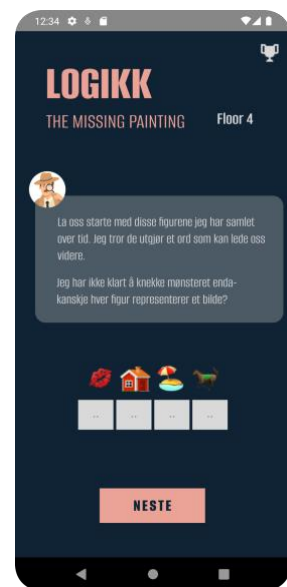
The tasks are of three different types. This feature in the application is only working in the prototype in Figma.





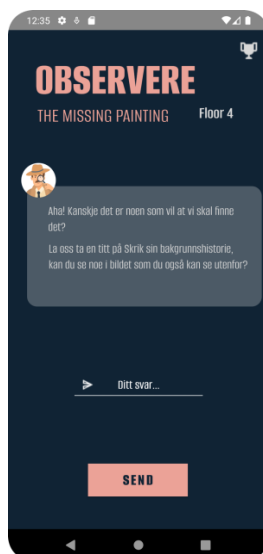
The detective is talking directly to the player. The first information the player gets is a backstory to understand what the case is about

The first usertask is a logical task. They have to find a word through figuring out which painting the emoji represents and insert the first letter of the painting's name. In this task the answer is KRIG, to get the user to think about WW2.



The detective continues filling in the story, to get the user on the right path. The next task is about interpretation, and the user has to find the picture in the exhibition that correlates with the task and scan that picture with the camera function. When they have answered correctly on the last task, they will get an answer from the detective, and move on to the next task.

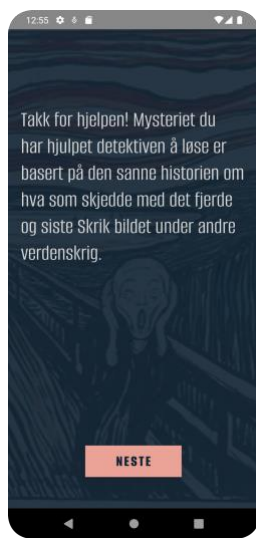
The next task is based on a substitution cipher, which is based on converting numbers to letters. So in this case the correct answer will be SKRIK.



After cracking the substitution cipher, the game goes over to an “observation” task. This task is about observing MUNCH’s surroundings in order to involve the physical space. The user is supposed to look at the background of “Skrik” and compare it to what is outside the museum. The correct answer will therefore be Oslofjorden.

After the user has guessed the right element of the physical surrounding, the fifth task begins, which is about solving a riddle. The user gets a clue that the picture is similar to Skrik and the four letters AIBG, which they have to translate using the same substitution cipher as one of the previous tasks. After encrypting the cipher code, they will get 1927, which is the year the painting “*Pikene på broen*” was created. To give an answer they scan the painting, the same way they did in the second task.

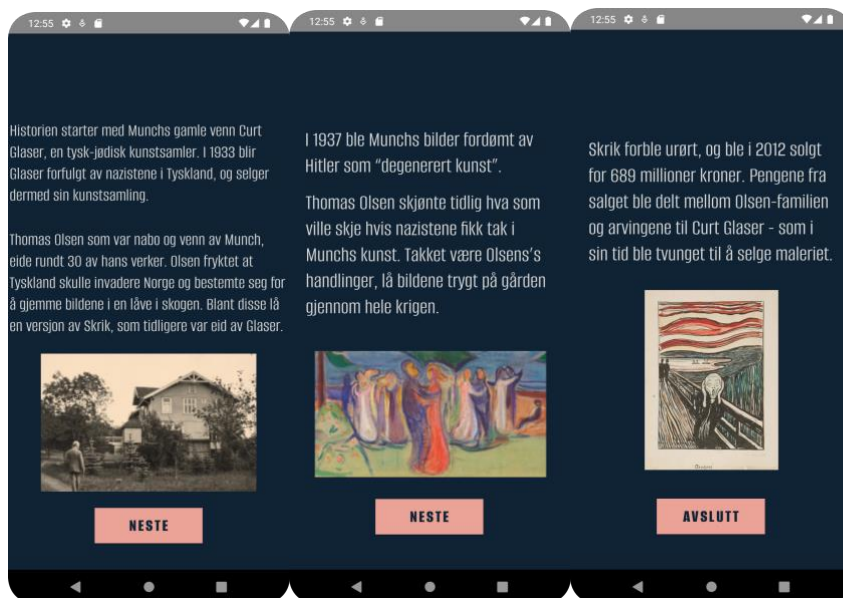




The last task is again based on typing the right answer. The detective gives information about Munch's friends and reveals that the neighbor is important to the story. They have to type the name of one of the paintings they found in the first task, because they might remember that there was a red house in the painting.

2.1.4 Endstory

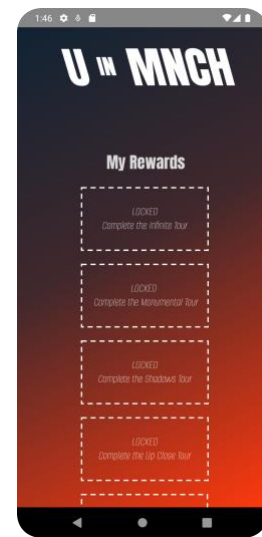
The mystery has now been solved and the user has access to the real story. It is revealed in a format resembling a social media story. This format makes it easier for the user to read the story for as long as they want and click back and forth.



2.1.5 Rewards

Having completed the tour, the user can unlock a reward.

Every reward belongs to a separate game.



2.2 Setup development environment

Download list:

Github repository: https://github.com/ChrisGloom1/porto_u_in_mnch

Docker for MacOS: <https://docs.docker.com/desktop/install/mac-install/>

Docker for Linux: <https://docs.docker.com/desktop/install/linux-install/>

Azure Data Studio & MS SQL Server: <https://learn.microsoft.com/en-us/sql/azure-data-studio/download-azure-data-studio?view=sql-server-ver16>

Visual Studio Code: <https://dotnet.microsoft.com/en-us/download>

.NET SDK: <https://dotnet.microsoft.com/en-us/download>

Android Studio: <https://developer.android.com/studio>

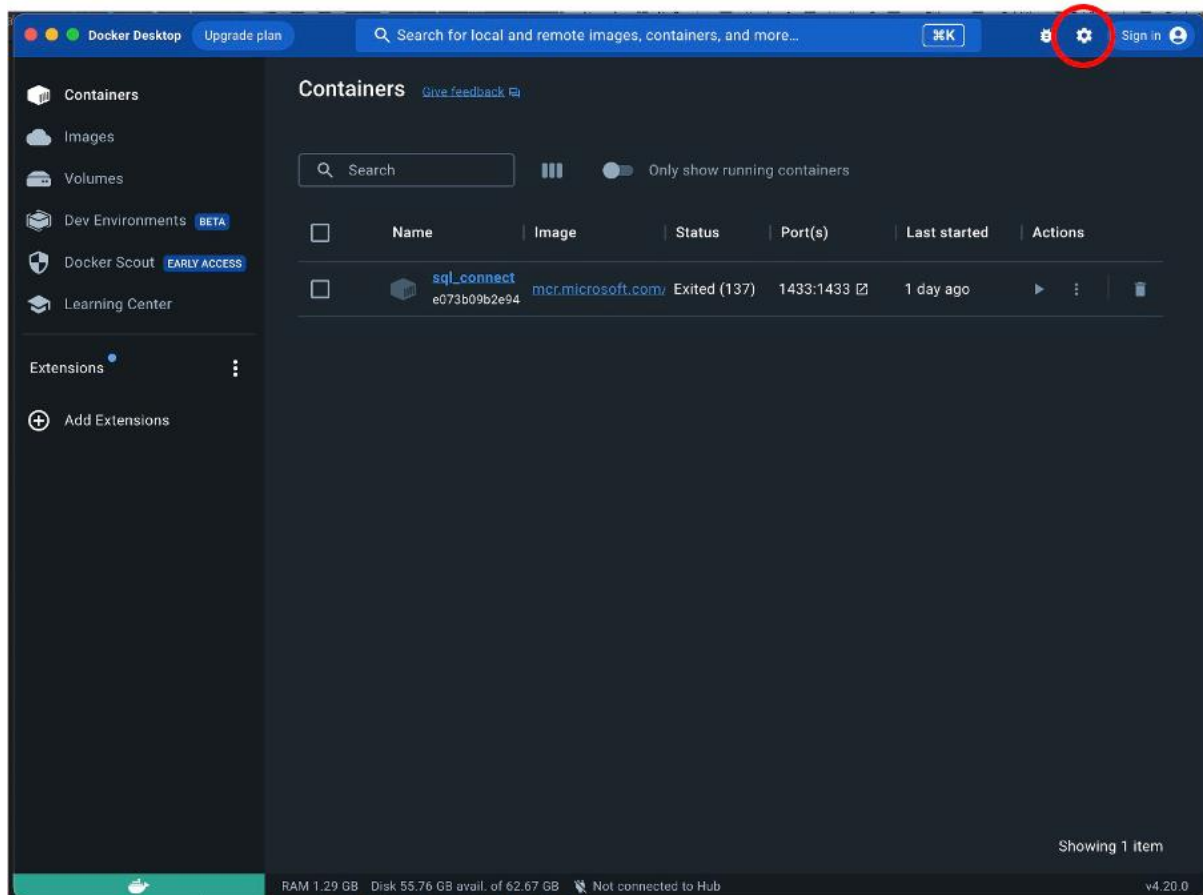
If the environment is to be configured on Mac or Linux, you would have to install Docker. In Docker we create a container for us to be able to login with SQL credentials. The credentials will be set when creating and running the container for the chosen Image.

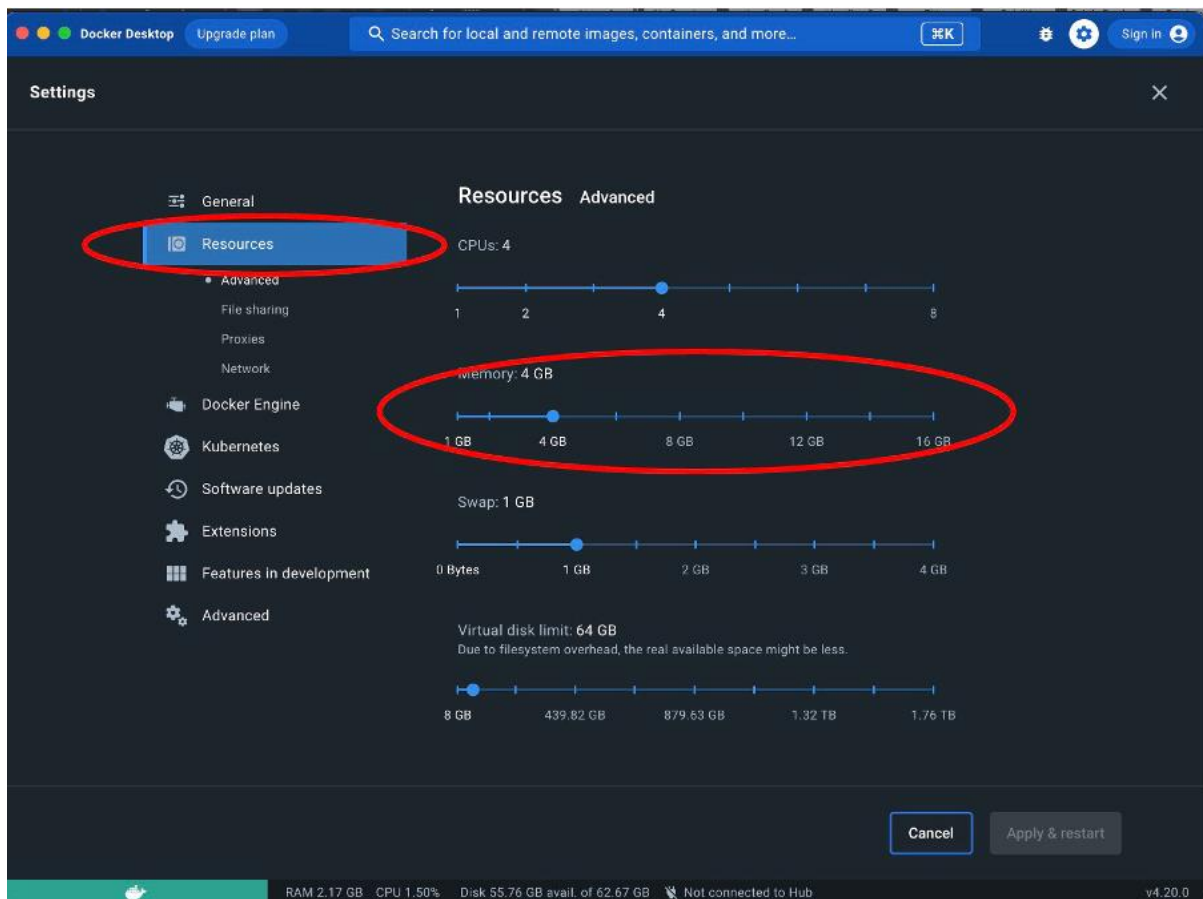
If the environment is to be configured on Windows, you can use your Microsoft credentials to login to Azure Data Studio/MS SQL Server. That means that there is no need to install Docker to get the environment up and running correctly.

Install and open Docker Desktop.

Choose to install docker with the regular setting and skip the roles and uses.

Open Docker and increase RAM allocation for docker. We've used 4GB which is enough for our project.





Open a terminal and insert the following command to create and run a container for the Image:

```
docker run -e "ACCEPT_EULA=1" -e "MSSQL_USER=SA" -e "MSSQL_SA_PASSWORD=Porto_Munch9" -e "MSSQL_PID=Developer" -p 1433:1433 -d --name=sql_connect mcr.microsoft.com/azure-sql-edge
```

- "MSSQL_USER=SA" sets the username to "SA".
- "MSSQL_SA_PASSWORD=Porto_Munch9» sets the password to "Porto_Munch9" for the user "SA". If the user is changed, the username in the command needs to be changed accordingly (teal highlight).
- The name of the container is sql_connect (purple highlight).

Check if the container is running:

```
docker container ls -a
```

```
(base) christopherglommenandresen@Christophers-MacBook-Air ~ % docker container ls -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS              PORTS          NAMES
e873b89b2e94   mcr.microsoft.com/azure-sql-edge    "/opt/mssql/bin/per...  12 days ago   Exited (137) 50 minutes ago           sql_connect

(base) christopherglommenandresen@Christophers-MacBook-Air ~ % docker container ls -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS              PORTS          NAMES
e873b89b2e94   mcr.microsoft.com/azure-sql-edge    "/opt/mssql/bin/per...  12 days ago   Up 5 seconds       1401/tcp, 0.0.0.0:1433->1433/tcp       sql_connect
```

If the status of the container is “Exited” the container is not running and needs to be started to connect to Azure Data Studio/MS SQL Server. See the command below to start the container.

Start the container called sql_connect:

docker start sql_connect

Stop the container called sql_connect – Please note that the container will run even if you close the terminal/desktop application:

docker stop sql_connect

Install and open Azure Data Studio & MS SQL Server.

Connect to localhost where you change the parameters in red squares below. Remember that you’d need to have the Docker container running for this to work. The username is “SA” and the password is “Porto_Munch9” unless this was changed in the steps above.

Connection Details

Connection type	Microsoft SQL Server
Input type	☒ Parameters ☐ Connection String
Server *	localhost
Authentication type	SQL Login
User name *	SA
Password	
	☐ Remember password
Database	<Default>
Encrypt ⓘ	Mandatory (True)
Trust server certificate ⓘ	True
Server group	<Do not save>
Name (optional)	

Advanced...

If you're running on a Windows, you can use the following configuration:

Connection Details

Connection type	Microsoft SQL Server
Input type	☒ Parameters ☐ Connection String
Server *	localhost
Authentication type	Windows Authentication
Database	<Default>
Encrypt ⓘ	Mandatory (True)
Trust server certificate ⓘ	True
Server group	<Do not save>
Name (optional)	

Advanced...

When connected you should run the SQL seed attached (MUNCH-seed.sql) or by copying the following text and insert in Azure Data Studio after connection is established:

```
-- Create the MUNCH database
CREATE DATABASE MUNCH;
GO

-- Use the MUNCH database
USE MUNCH;
GO

-- Create the MUNCH schema
CREATE SCHEMA MUNCH;
GO

-- Create the Users table in the MUNCH schema
CREATE TABLE MUNCH.Users (
    UserId INT IDENTITY(1,1) PRIMARY KEY,
    Email VARCHAR(50),
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Password VARCHAR(50),
    FullName AS (FirstName + ' ' + LastName),
    DateOfBirth DATE,
    CreatedAt DATETIME
)
```

```

);
GO

-- Create the Infinite_v1 table in the MUNCH schema
CREATE TABLE MUNCH.Infinite_v1 (
    ID INT IDENTITY(1,1) PRIMARY KEY,
    userID INT,
    task1 BIT,
    task2 BIT,
    task3 BIT,
    task4 BIT,
    task5 BIT,
    task6 BIT,
    FOREIGN KEY (userID) REFERENCES MUNCH.Users(UserId)
);
GO

-- Insert data into the Users table
INSERT INTO MUNCH.Users (Email, FirstName, LastName, Password, DateOfBirth, CreatedAt)
VALUES
    ('test@gmail.com', 'Test', 'Testsson', 'ProgrammingIsCOOL', '1995-06-09', GETDATE());
GO

-- Insert data into the Infinite_v1 table
INSERT INTO MUNCH.Infinite_v1 (userID, task1, task2, task3, task4, task5, task6)
VALUES (1, 0, 0, 0, 0, 0, 0);
GO

```

(see attachment 00_sqlseed.sql)

Install both Visual Studio Code and the .NET SDK. Make sure to include the “.NET desktop development” and “ASP.NET and web development”. This will ensure that you have all the necessary tools and templates to create web APIs with C#.

As packages aren’t automatically installed on other systems, you’d have to add the following packages in the WebAPI directory through the terminal:

Dotnet add package dapper

Dotnet add package Microsoft.Data.SqlClient

If you’re running on a Windows, you’d need to change the ConnectionStrings inside appsettings.json in the WebAPI to the following string:

```
"ConnectionStrings": {
```

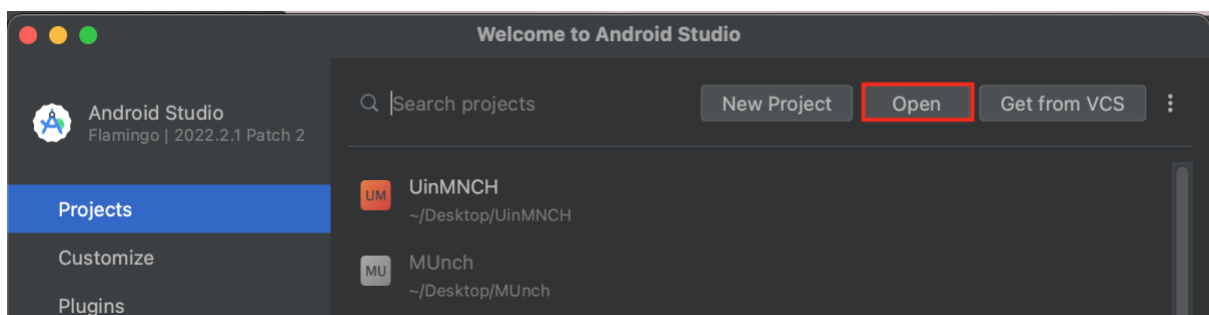


```
"DefaultConnection" : "Server=localhost; Database=MUNCH;  
Trusted_Connection=true;TrustServerCertificate=True;  
},
```

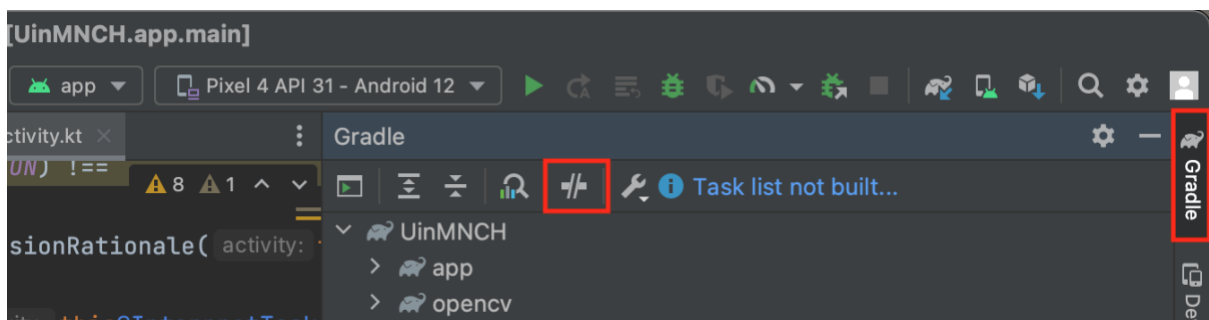
The only thing missing now is downloading Android Studio and running the app through Android Virtual Device (AVD).

2.3 How to test the application

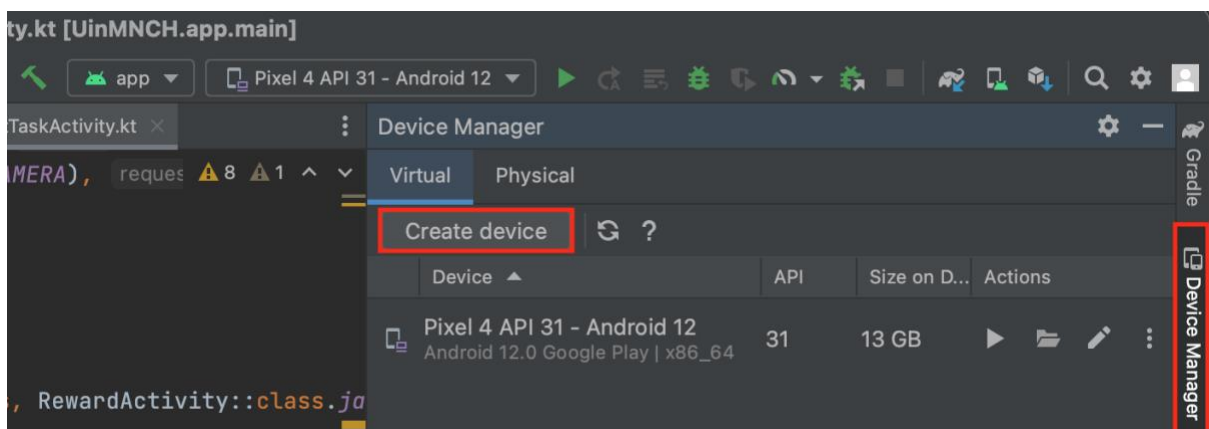
To test the application it is necessary to have Android Studio installed. The project directory is uploaded to GitHub repository and should be downloaded from there. Next, the project directory has to be opened in Android Studio:



Then, is necessary to toggle online mode in the Gradle tab:



Once it's opened, it is necessary to install a virtual device in DeviceManager tab:



We have used Pixel 5 API 31 for testing so we recommend the same device.

After successful installation of the virtual device, the app is ready to be built. Click on the green arrow:



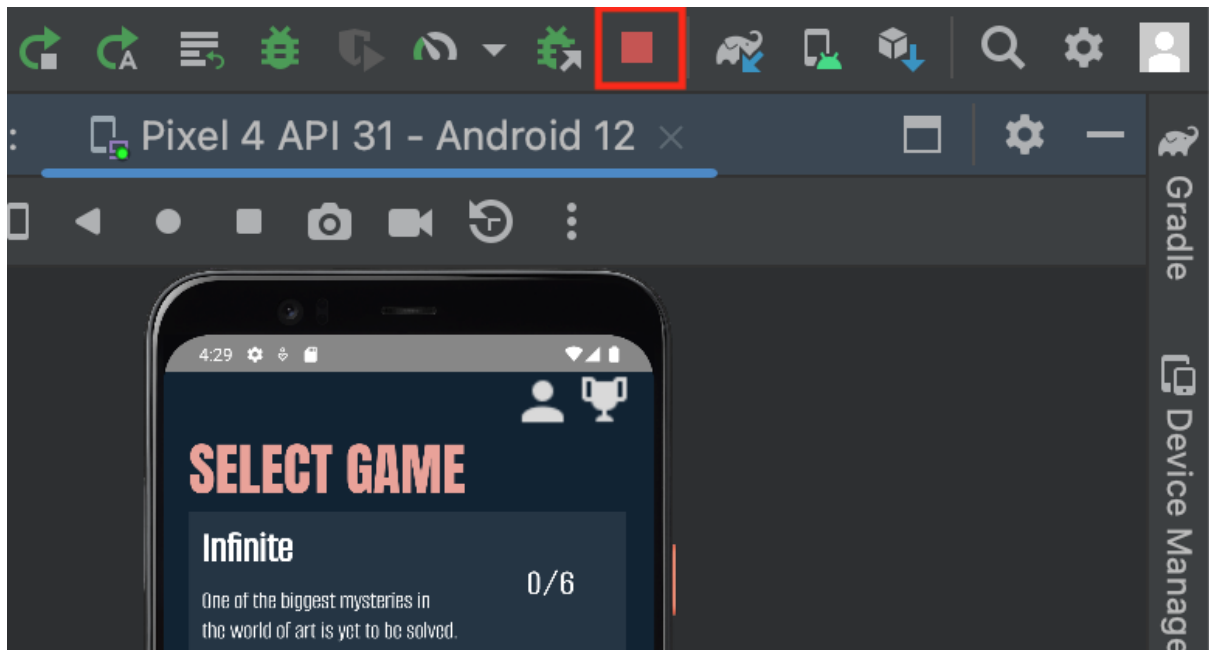
When the application is built and launched in the emulator, allow internet and camera permissions once the permission dialog appears on the screen.

To proceed in the application, click on the LOGIN button without providing any input in the text fields. Then, in the tasks which require the text input from the user as answer, type in respectively:

- *k r i g* (each letter in separate text field respectively) on LOGIKK task
- *skrik* on KODEN task
- *oslofjorden* on OBSERVERE task
- *rod villvin* on AVSLØRINGEN task

In the tasks which require taking a photo, click on the camera button and click a tick under the virtual camera view. When the result screen gets displayed, click on the “NEXT” button.

After completing the tour and coming back to tour list screen, terminate the process by clicking on the red rectangle:



3 Technical choices

3.1 Programming language

C# was the chosen programming language for the backend in this project as most of the group members had created a WebAPI in web development in the third semester. We also used the .NET 7 framework with a couple packages added to it to send queries to the database. There was some delay in finalizing the backend, as it had to be relearned again during the sprints.

There was some discussion around how we would create the app, but we ended up with Kotlin/Android Studio as the programming language used for the frontend. We had some members of the group that had Android Programming in third semester, and it has a very similar syntax as Java which everyone is familiar with. We coded up the design in Android Studio with XML, and connected everything together in the end to have a functional app.

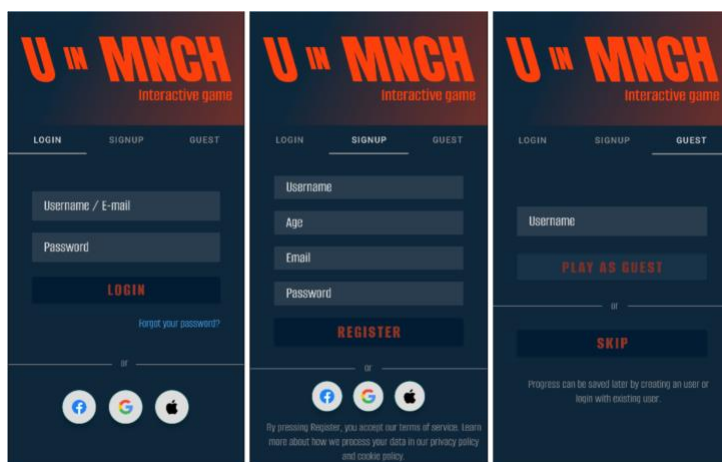
3.2 Database

Our database is in Azure Data Studio/MS SQL Server. The reason we went for this instead of MySQL or SQLite is that we had access to a WebAPI course where this was included. As the

WebApi was created the database was created at the same time. It might've been easier to set up a different database, but as Azure is a widely used application in working places it increased relevant learning in the sprints. We only had two tables for the MVP containing user data and a scoreboard for one tour in the museum. We believe the structure of the tables in the database could be normalized. As none of the members of the groups feels very stable in this process we decided to make the database functional for the MVP and the database normalization could be done at a later stage in the development.

3.3 Login system

The entire login system is visible on a singular page. This is achieved by having the common elements (logo, subtitle and tab-bar (login, signup and guest)) in the main view, and then using Kotlin to change the designated area with the fragment belonging to the chosen tab.



(Comparison of the fragments.)

It's always possible to continue in the application by pressing the login button, without actually connecting to the database. The player will be given id 1. We were unsure how to program the solution to store which user is logged in throughout the whole app. So working Login-button, without actual log in, is to ensure that we can showcase our solution.

However, we have programmed the entire function for log in:

```
// Log in with the user data
private fun loginUser(email: String, password: String) {
    apiService?.getUsers()?.enqueue(object : Callback<ArrayList<User>> {
        override fun onResponse(
            call: Call<ArrayList<User>>,
            response: Response<ArrayList<User>>
        ) {
            // Handle successful response
            if (response.isSuccessful) {
                val userList = response.body()
                userList?.let { it: ArrayList<User> }
                val user = it.find { it.email == email && it.password == password }
                // Handle successful log in
                if (user != null) {
                    val intent = Intent(activity, ToursActivity::class.java)
                    startActivity(intent)
                }
                // Handle log in failure
            } else {
                Toast.makeText(requireActivity(), text: "Wrong e-mail or password", Toast.LENGTH_LONG).show()
            }
        }
    })
}
```

Ideally, the next activity would be launched only on successful login.

The second method is through signup. It depends on the application having a successful API-response and internet connection to register, or else a notification will signal that it's not working. This is necessary so the inputs can be stored as a new account in the database. The sign up function is programmed in such a way that it won't launch new activity (tour list) if the user creation is not successful. The function makes a request createUser to the database, and this request is already defined in the ApiService interface.

```

// Make a HTTP request to add a new user
private fun addNewUser(user: CreateUserRequest) : Boolean {

    // Call an user creation function from API Interface
    val callAddNewUser: Call<ResponseBody> = apiService.createUser(user)

    // Local variable to determine the sign up result
    var success = false

    callAddNewUser.enqueue(object : Callback<ResponseBody> {

        //Handle successful response
        override fun onResponse(call: Call<ResponseBody>, response: Response<ResponseBody>) {
            success = if (response.isSuccessful) {
                Log.i( tag: "New-user-response", msg: "Successful")
                true
            } // Handle API call failure
            } else {
                val errorCode = response.code()
                Log.i( tag: "New-user-response", msg: "$errorCode")
                Toast.makeText(requireActivity(), text: "Sign up failed", Toast.LENGTH_LONG).show()
                false
            }
        }

        // Handle network request failure
        override fun onFailure(call: Call<ResponseBody>, t: Throwable) {
            Log.i( tag: "New-user-response", msg: "Response failure ${t.message}")
            Toast.makeText(requireActivity(), text: "An error occurred", Toast.LENGTH_LONG).show()
            success = false
        }
    })
}

return success

```

Request for new user creation and handling successful or failed response

Playing as a guest is a non-functioning method, the same goes for the alternative way of logging in and registering with a second party account (Facebook, Google and Apple). The additional options are included to give easier and faster access, thus better the experience of the application and reduce the threshold to use it.

3.4 Functionalities

The application is programmed in accordance with the principles of object oriented programming.

The core functionality are tasks of two types. The first type is based on typing in an answer in an input field. The second one is based on taking a photo with the phone camera. These two types of quests alternate during the game.

Both task types have their own task model class with constructor arguments together with getters and setters for every argument.

```
class WriteTaskModel (
    private var taskNr: String, // For database update
    private var title: String, //The name of the task
    private var subTitle: String, // The description of the tour the task belongs to
    private var content1: String, // Detailed task description in the first chat bubble
    private var content2: String, // Detailed task description in the second chat bubble
    private var answer: String // The correct answer
){}
```

Task model's constructor for tasks based on typing text in input field as an answer

```
// An universal task model that is based on picture scanning w/camera
class ScanTaskModel (
    private var id: Int, // From Munch's API
    private var taskNr: String, // For database update
    private var title: String, //The name of the task
    private var subTitle: String, // The description of the tour the task belongs to
    private var content1: String, // Detailed task description in the first chat bubble
    private var content2: String // Detailed task description in the second chat bubble
){}
```

Task model's constructor for tasks based on taking a photo as an answer

With these two models, adding new quests to the game is easy as it is only needed to create new instances of these model classes.

The task pool is specified in the Constants class. There are two array lists with model instances. Each list contains instances of a specific task type. Thus, further classes representing specific tasks fetch their content from the appropriate list from the Constants class.

```

object Constants {

    // Image URL - the image's ID value is in the middle of the query
    const val URL_1 = "https://munch.emuseum.com/apis/iiif/image/v2/" // The first part of Munch API's image url
    const val URL_2 = "/full/full/0/default.jpg" // The first part of Munch API's image url

    const val REQUEST_CODE = 200 // For the camera launching function

    // Create a list of tasks of type ScanTaskModel
    fun scanTaskList(context: Context): ArrayList<ScanTaskModel>{
        val scanTaskList = ArrayList<ScanTaskModel>()
        val task1 = ScanTaskModel(
            id: 4746, // ID found in Munch's API
            taskNr: "task2",
            title: "Tolke",
            subTitle: "The missing painting",
            context.getString(R.string.speech_tolke1),
            context.getString(R.string.speech_tolke2)
        )
        scanTaskList.add(task1)
        val task2 = ScanTaskModel(
            id: 5361, // ID found in Munch's API
            taskNr: "task5",
            title: "Gåten",
            subTitle: "The missing painting",
            context.getString(R.string.speech_gaaten1),
            content2: ""
        )
        scanTaskList.add(task2)
        return scanTaskList
    }
}

```

Constants object class: List of tasks based on taking a photo with phone camera

Every specific task has its own separate activity (a class that creates a new window where the UI is placed). Every activity is connected to a corresponding xml layout file through binding. *View Binding* is a feature which makes it easier to interact with xml layout elements, because the binding class instance created for the desired xml layout contains direct references to all the elements in the layout that are given IDs.


```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    // assigning layout to the activity
    binding = ActivityCodeBinding.inflate(layoutInflater)
    setContentView(binding?.root)

    // assigning xml elements to variables
    btnNext = binding?.buttonNext
    btnRewards = binding?.btnRewards
    inputAnswer = binding?.inputAnswer
}
```

Example. Binding an activity to a layout and setting this layout as the content view. Then fetching layout elements by ID and assigning them to variables. The variables are declared upper in the code, yet before the onCreate function which builds the activity.

In the activity, the necessary layout elements are referred to through variables. Next, they are given content by assigning them to the task arguments from the relevant Constants task list (through setupTaskView() function called on activity launching).

```
// setting up the task view
@RequiresApi(Build.VERSION_CODES.M)
private fun setupTaskView() {

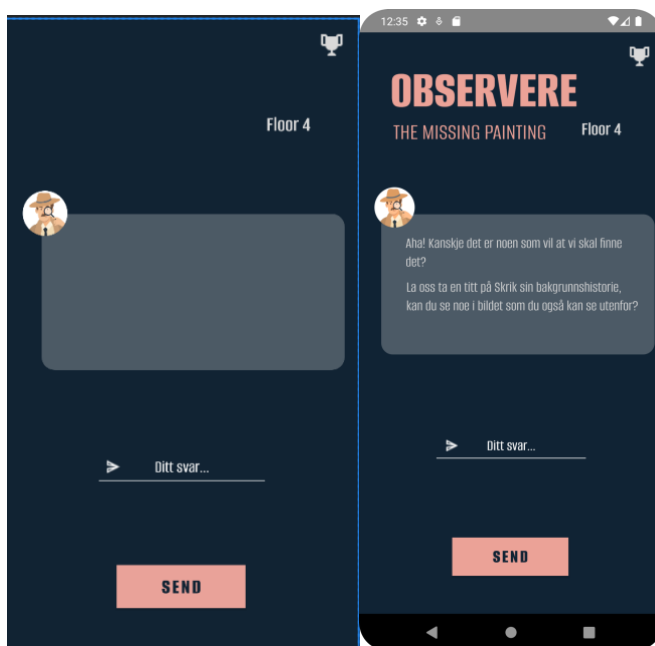
    // assigning xml elements to corresponding variables
    tvTaskTitle = binding?.tvTaskTitle
    tvTaskSubTitle = binding?.tvTaskSubTitle
    tvTaskContent1 = binding?.tvTaskContent1
    tvTaskContent2 = binding?.tvTaskContent2
    cipherLayout = binding?.cipherLayout

    // making cipher element invisible for this particular task
    cipherLayout?.visibility = View.INVISIBLE

    // assigning the task's arguments to the layout variables
    tvTaskTitle?.text = taskList!![currentTaskPosition].getTitle().uppercase()
    tvTaskSubTitle?.text = taskList!![currentTaskPosition].getSubTitle().uppercase()
    tvTaskContent1?.text = taskList!![currentTaskPosition].getContent1()
    tvTaskContent2?.text = taskList!![currentTaskPosition].getContent2()
    ivStoredImg = binding?.ivStoredImg
    ivStoredImg?.visibility = View.INVISIBLE
}
```

Example. Referring to layout elements first and then assigning task arguments to them. Current task position is the task's index in the Constant's task list. Both this number and the task list itself are initialized as a global variable higher in activity code.

This way, the empty layout elements get filled with task content:



Example. Before and after compilation

and moreover, the layout may be reused in new tasks of the same kind.

Each of the two task types has its own answer verification functionality. For the first kind (text input answer) it is naturally a much simpler function - a basic if-statement comparing user answer to the correct answer fetched from a relevant task instance in Constants:

```
// check if user's answer is correct and if so, update score and launch the next task's activity
btnNext?.setOnClickListener { it: View!
    val answer = inputAnswer?.text.toString().trim().lowercase()
    if (answer == taskList!![currentTaskPosition].getAnswer()) {
        Constants.updateTask( taskAndNumber: "task3", solved: true, userId: 1)
        val intent = Intent( packageContext: this, ObserveTaskActivity::class.java)
        startActivity(intent)
    }
}
```

Example of text answer verification

For the next type, the camera scanning task, there is created a whole separate object class with necessary functions for answer verification. These functions are created for performing image comparison and they implement the OpenCV library. OpenCV is complex and thus may seem excessive for this application, but it is the only such library that works with Android. There were a few much smaller GitHub libraries created to perform this kind of image comparison, but they are not adapted to work with Android and therefore resulted in import failure. In the image comparison object class, there is the main function `compareImages` returning a Boolean: true if the user has scanned the right picture, or false if wrong. There are multiple helping functions for this one. The comparison is performed in the following order:

1. In the task's activity, Glide library loads an image (that is the correct answer) from its URL into the task layout's `ImageView`. This picture remains invisible in the layout, but is stored there for later comparison purposes. This process is a part of the `setupTaskView` function.

```
// fetching the right picture from Munch API for comparison purposes
val id = taskList!![currentTaskPosition].getId()
val url = Constants.URL_1 + id.toString() + Constants.URL_2
if (Constants.isNetworkAvailable( context: this)) {
    ivStoredImg?.let { it: ImageView
        // Glide library to load a picture from URL into a xml layout element
        Glide.with( activity: this@InterpretTaskActivity) RequestManager
            .load(url) RequestBuilder<Drawable!>
            .into(it)
    }
}
```

- Both the stored correct answer picture and the photo taken by the user get converted into Bitmap. User's photo is cast to Bitmap, while the stored correct answer picture is converted by the helping function convertImageViewToBitmap.
- Then the compareImages function is called on, taking these two bitmap variables as arguments.

```
// storing the photo taken by the user and comparing it with the correct answer picture
@RequiresApi(Build.VERSION_CODES.M)
override fun onActivityResult(requestCode: Int, resultCode: Int, data: Intent?) {
    super.onActivityResult(requestCode, resultCode, data)

    if (resultCode == Activity.RESULT_OK && requestCode == Constants.REQUEST_CODE && data != null) {
        capturedImg = data.extras?.get("data") as Bitmap

        if (capturedImg != null) {
            // converting the image from API to bitmap
            val storedImageBitmap = ivStoredImg?.let { it: ImageView
                ImageComparison.convertImageViewToBitmap(
                    it
                )
            }

            // comparing the two images
            if (storedImageBitmap != null) {
                isSimilar = ImageComparison.compareImages(capturedImg!!, storedImageBitmap)
                displayResultDialog()
            }
        }
    }
}
```

Cast of the user's photo (capturedImg) to Bitmap and the API image (ivStoredImg) conversion to Bitmap. Then calling the comparison function (isSimilar) with these two as arguments. Finally, display a dialog with the comparison result.

- The compareImages function converts the images into Mat type using the helping function bitmapToMat. Mat type is needed for OpenCV methods.
- Then the compareImages function converts these two Mats into grayscale and resizes them using OpenCV classes.
- Next, the function computes hash values for these two Mats using the helping function computeHash.
- Having these two hash values ready, the function calls the helping function computeSimilarity to determine the similarity percentage.
- Once the similarity percentage is returned, the function compares it with the predefined threshold value and returns false if the similarity is lower than the threshold (which is 80%).

```

object ImageComparison {

    // Final comparison if two images
    fun compareImages(userImage: Bitmap, storedImage: Bitmap): Boolean {...}

    // helping function to compute image hash for later comparison
    fun computeHash(image: Mat): String {...}

    // Compute similarity of two hashes
    fun computeSimilarity(hash1: String, hash2: String): Double {...}

    // Helping function to convert Bitmap to Mat required for OpenCV methods
    fun bitmapToMat(bitmap: Bitmap): Mat {...}

    // Helping function to convert Image to Bitmap required for later Mat conversion
    fun convertImageViewToBitmap(imageView: ImageView): Bitmap? {...}
}

```

Object class with the main function for image comparison (compareImages) and necessary helping functions below it.

When a task is solved correctly, its corresponding table cell in the database gets updated from false to true. Because the function that updates the score is the same for all the tasks, it is created in Constants. Every task activity calls on it in case of correct answer.

```

// Update task score in the database
fun updateTask(taskAndNumber: String, solved: Boolean, userId: Int) {
    val request = UpdateScoreboardInfinitev1(taskAndNumber, solved, userId)
    val retrofit = Retrofit.Builder()
        .baseUrl(DB_URL)
        .addConverterFactory(GsonConverterFactory.create())
        .build()
    val apiService = retrofit.create(ApiService::class.java)
    val call = apiService.updateScoreInInfinitev1Table(request)

    call.enqueue(object : retrofit2.Callback<Void> {
        override fun onResponse(call: Call<Void>, response: retrofit2.Response<Void>) {
            if (response.isSuccessful) {
                Log.d("API Response", "Scoreboard updated.")
            } else {
                Log.d("API Response", "Unable to update scoreboard")
            }
        }

        override fun onFailure(call: Call<Void>, t: Throwable) {
            Log.e("API Response", "Request failed with error: ${t.message}")
        }
    })
}

```

The entire updateTask function in Constants object class

The function takes in arguments the taskNr (argument in each task model's constructor), because it's also the column name in the score table in the database. The next argument is of course boolean true, and the last one is userId. The userId should ideally be fetched from the database through the log in function, but we didn't manage to solve how the app can store the logged-in user's ID throughout all the contexts in the application. Therefore, wherever the update function is called on, "1" is passed as userId argument. In order to connect to the database, the function uses the Retrofit library and ApiService interface.

In the API Service interface, there are specified requests which the app can send to the database. As shown in updateTask function, one of these requests is UpdateScoreboardInfinitev1. Other functions in the interface are requests used for log in, sign up and point sum retrieval purposes.

```

interface ApiService {
    @GET("TestConnection")
    fun testConnection(): Call<String>

    @GET("GetUsers") // Retrieve the list of users
    fun getUsers(): Call<ArrayList<User>>

    @POST("AddNewUser") // Create new user in db
    fun createUser(@Body request: CreateUserRequest): Call<ResponseBody>

    @PUT("UpdateScoreInInfinite_v1Table") // Update task result in case of correct answer (false -> true)
    fun updateScoreInInfinitev1Table(@Body request: UpdateScoreboardInfinitev1): Call<Void>

    @GET("GetSumOfScoreBoardInfinite_v1") // Retrieve the sum of all right answers in order to unlock rewards
    fun getPointSum(@Query("userId") userId: Int): Call<Int>
}

```

ApiService Interface and its functions database requests

If all tasks are solved, a coupon gets unlocked on the rewards page. This is done in the Reward Activity. Its function `fetchPointSum` changes layout variables according to points retrieved from the database. If the point sum is equal to the number of tasks (meaning all tasks got solved), the coupon element in the layout changes its text from “Unlocked” to the desired coupon content.

```

@RequiresApi(Build.VERSION_CODES.O)
private fun fetchPointSum(userId: Int) {
    taskList = Constants.scanTaskList(context: this)
    tvFreeCoffeeCoupon = binding?.freeCoffeeCoupon
    val typefaceUnlocked = resources.getFont(R.font.anton)
    val typefaceLocked = resources.getFont(R.font.alumni_sans)
    val retrofit = Retrofit.Builder()
        .baseUrl(Constants.DB_URL)
        .addConverterFactory(GsonConverterFactory.create())
        .build()
    apiService = retrofit.create(ApiService::class.java)
    val call = apiService.getPointSum(userId)
}

```

First, the function connects to database and retrieves the point sum

```

call.enqueue(object : Callback<Int> {
    @SuppressWarnings("SetTextI18n")
    override fun onResponse(call: Call<Int>, response: Response<Int>) {
        if (response.isSuccessful) {
            val pointSum = response.body()
            pointSum?.let { it: Int
                // Check if point sum equals task list size
                if (it == taskList!!.size) {
                    // Unlock the reward by changing the textView
                    tvFreeCoffeeCoupon?.text = "@string/coupon_free_coffee"
                    tvFreeCoffeeCoupon?.textSize = 12F
                    tvFreeCoffeeCoupon?.typeface = typefaceUnlocked
                } else {
                    tvFreeCoffeeCoupon?.text = "@string/locked_infinite_tour"
                    tvFreeCoffeeCoupon?.textSize = 18F
                    tvFreeCoffeeCoupon?.typeface = typefaceLocked
                }
            }
        }
    }
})

```

Then, in case of successful database response, the function performs comparison of the point sum to the number of tasks. If it's equal, the coupon layout changes, or else it stays the same (locked).

3.5 Improvements

It would be better to use a different kind of database where it wouldn't be as complicated to set up the development environment on MacOS or Linux. Using SQLite connected to the WebAPI would most likely have been a better solution. Best practices aren't taken into consideration for the MVP as no one in the group felt skilled in this area, so there's most likely a lot of improvements to ensure the code's readability, maintainability and performance. The database could also be normalized to get an improved database.

When we ran the application all together for the first time, it crashed because some of the elements in the xml made in constraint layout were missing necessary positions. Also, some elements changed position because it wouldn't work without being inside a linear layout. Until then everything seemed fine, as that didn't happen when the files were in their own project and tested in the emulator. These common issues would've been avoided by sticking to linear layout from the start. Another advantage is that there would have been less code since it wouldn't be needed to specify in detail the placement of elements and adding two layouts.

Also, the xml for the task pages could be improved by structuring them as the login system, with a shared main view and changing fragments. This will ensure that identical layout and elements have a fixed placement and stay the same throughout the application, and result in less repetitive code.

4 Theoretical choices

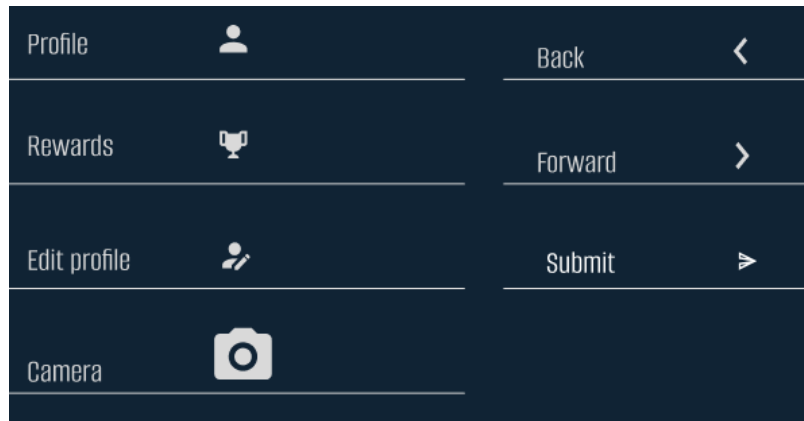
4.1 Design

Two of the candidates in our group are from Interactive Design, so it became important for us to create a consistent and innovative solution. However, the other candidates were included in this process to learn something new. We intended our design to be modern for our target audience, and we spent a lot of time during our sprint to develop and refine the user experience. While developing the design, we always considered the product owner's opinions and used munchmuseet.no as a reference and foundation of the structure.

Our goal was to make a user-friendly and interactive app for MUNCH that can be enjoyed by all audiences. It should be a fun experience that would attract more people and encourage a younger audience to revisit. Our main audience is 18-25, so all the important decisions were made with that in mind, but we would love to add more functionality for a larger audience in the future with more time.

4.1.1 Icons

Since the icons we have used are self-explanatory enough for the users to understand the next step, there was no need to use a lot of text for explaining. We decided to maintain a clear contrast color to the background, so the user easily recognises a clickable icon. The icons we have used are from Material Design Icons(See figure 1), which is a Figma Plugin. (Icons8 Team, 2023)



(Figure 1)

4.1.2 Color choices

The colors we chose were already used on Munchmuseet.no, since we knew they would appeal to the product manager's vision(See figure 2). Each exhibition has their own contrast colors, but we created tasks and programmed the solution for "Infinite", which has light pink text colors with a dark blue background. We were satisfied with the "Infinite" colors and therefore chose to use this in other pages of our solution, as it was comfortable to look at. The header is in the classic MUNCH-red color, and we combined this color with the dark blue background on the profile page and rewards page.

Primary



Secondary



(Figure 2)

4.1.3 Fonts

We used Anton for Logo and h1, which resembles the same font as munchmuseet.no. The font is naturally bold and sharp and matches our other font (Alumni Sans). Alumni sans is used in h2 and other descriptions. It has the same sharp edges and style as Anton, but since it's more flexible we can adjust the boldness to our preference on the different paragraphs/headers. The logo is also modified to reverse cursive in Photoshop. (See figure 3)



(Figure 3)

4.2 UI Choices

- We have made buttons, fonts, and color-templates in Figma for a consistent and clean app-design.
- A visual progress indicator when you select the game, that shows which tasks the user has completed.
- Implemented inspiration from Instagram stories “progression bar”. If the user wants to get more information about the backstory of the Infinite-mystery. This is added to appeal and make associations for the target audience.

4.3 UX Choices

- Simple and intuitive navigation that is self explanatory.
- Gamification is used as one of the most valuable ways to get users to download the app to experience engagement.
- Making a complete game experience for “Infinite” within the gamification.
- User testing and research (*see attachment 01_userdata.xls*)

4.4 Future UX Solutions

- A settings button in the registering process, before they choose exhibitions. Would give a user the opportunity to change settings and preferences early. Also give users an opportunity to edit their profile settings later.

- More improvements for users with colorblindness, impaired vision. Include lightmode, darkmode and larger text. Add these options in the settings.
- Clearer transitions and slowed down animations on buttons, to give the user a confirmation of their actions and new changes.
- Text underneath icons to make buttons more understandable for an older audience.
- Possibility to change the language in the app to include tourists that visit MUNCH.

5 Prototype

5.1 Design tools

In the Design Sprint workshop, we worked on our Miro board to lay a foundation for this project. This started with defining challenges, goals and focus areas to get some basics for making concept sketches. We voted for one of the suggestions, and made a user test flow and storyboard based on it.

For prototyping we wanted to use a tool the majority of the group was familiar with. Since most of the group had previously used Figma, this is the tool we ended up using.

Additionally we think Figma has a user friendly design in terms of functionality and design.

By having the majority of the group with previous knowledge of the tool, it was easier to give an introduction to those who hadn't used it before. This saved us time by not having any more of us than necessary to familiarize with a totally new design tool. This way everyone could easily contribute to the prototype process from the start.

5.2 Prototypes

5.2.1 Prototyping

The first prototype was based on the storyboard from Miro. We soon realized that the flow wasn't contributing to the main purpose of the application, and changed the focus to solely being leading to the tasks. This new starting point made it possible to later add less important pages such as scoreboards, ratings and rewards -

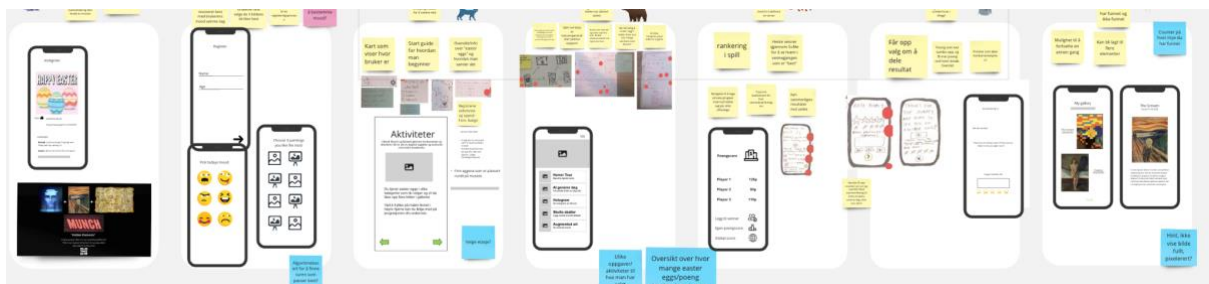
if we still wanted and had the time to enhance the application. Later on, this decision proved to be the basis of getting the feedback from usertesting and Product Owner that led to our final prototype and solution.

With the help of Figma, we were able to display our ideas in a way that made our concept come across clearer. The prototypes we made using Figma were a big help when intersecting through the different options. Having several prototypes resulted in faster decision making, as well as everyone having an impact on the solution. When prototyping, we added the most important details from design to function, keeping in mind that the final prototype would be a clear mockup for the group members coding the application. This also applied to the user test flow, as it's easier to test in Figma opposed to re-editing code.

The final prototype has been made with the intention of it being as realistic as possible. By using Figma we've been able to see and test the prototype on our phones, which made it easier to spot needed adjustments.

5.2.2 Wireframe

Our first wireframe was made in Miro, based on the user stories we developed in the Design Sprint. (See figure 4)

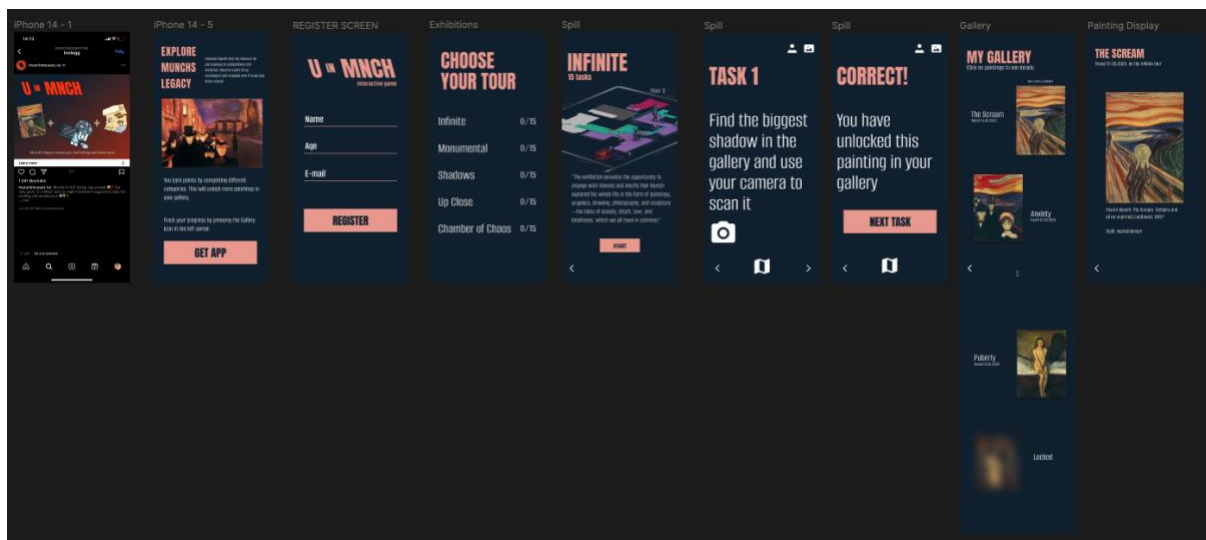


(Figure 4)

5.2.3 First prototype

The first prototype was developed in Figma, before our first Sprint Review with the Product Owner. We made changes from the Wireframe as we did

not see a clear correlation between the frames. We also added a more clear vision of how to choose a tour.



(Figure 5)

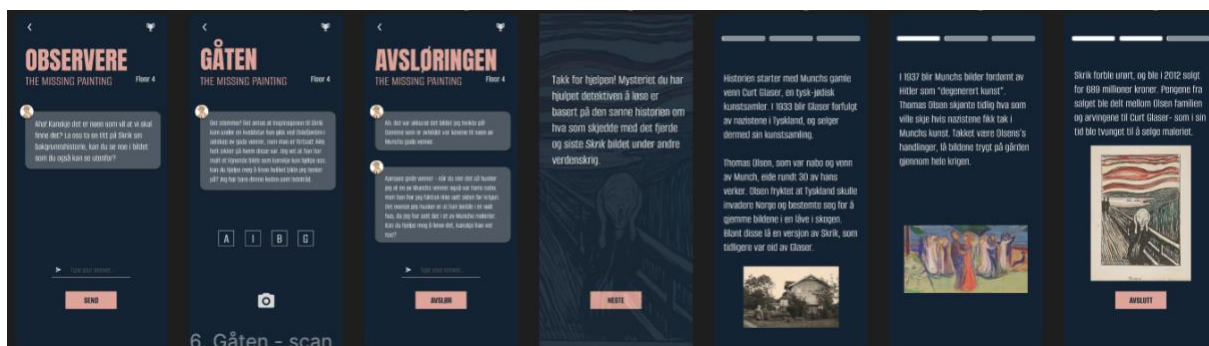
5.2.4 Final prototype

The final prototype we made was based on the feedback we got from PO, 26.05. We were told to remove the gallery feature, and focus on creating more creative tasks. We came up with the idea of having a reward system, instead of the gallery as an end goal for the user. After completing a game, the user gets a reward, and as an example we chose “get a free coffee”.

To make the tasks more engaging and creative, we chose to create a mystery the user had to solve. The tasks were made to give clues to the user, while having to think and explore the exhibition. To guide the user through the game, we added a detective. He introduces the tasks to the user in a message format. We decided this would be a fitting format based on our target group. We created 6 different types of tasks to help showcase our idea and concept to MUNCH.



Since we chose the concept of a mystery, we somehow had to explain the complete story to the user. To further engage with the target group, we wanted to implement another feature young people can relate to. Therefore we chose to display the story in a story format, inspired by Instagram stories. In this prototype version, we made a timer animation where the story automatically changes page when the time runs out. The user is displayed a progression bar, where they see how long the page will be displayed, before changing.



This was the final prototype we made in Figma, which is also the prototype we tried to replicate with programming.

6. Usertesting

6.1 Purpose of usertesting

To better understand our audience and get a clue on how well our prototype was, we decided to perform a user test. We wanted to find out if our design satisfied the users, how easy it was to navigate, if they understood the prototype, and if they enjoyed our concept. We started the usertest with an interview to learn more about their interest, which we could later use to analyze and compare their answers. Then, we gave them a few tasks where they would navigate their way through the app. Our goal was to find out how easily they performed the tasks and how much time they spent on the tasks. They were also allowed to ask for help during the test if they were stuck or ask questions. We preferred the users to speak their mind and talk us through their thought process. They were allowed to explore the app afterwards to understand it better. At the end of the test, we asked the users specific questions about the app that we wanted to learn more about. The questions were based on everything from the concept and tasks, to how user-friendly it was.

6.2 Execution

The tests were performed on 5 different users. Our goal was to find users in the age of 18-25, but with limited time, we had three users outside our age group in their 20's, beginning of 30's. However, they had similar interests to our age group with enough valuable data to make changes in our project. Because of their schedule, we had to arrange our tests in private where some were performed online and others met up in person. They tested the prototype on pc although the prototype was made for phones, but this decision was made as it would provide us more accurate data. During the usertest, we recorded their screen and sound, as well as their face. The user had to give their permission in advance. After the tests were completed, we sat down together and analyzed the footage, facial expressions and wrote down their answers in an excel file. We looked for similarities, differences and agreed on the conclusion of the test. The beginning of the test began with an interview to know the user, then the test began and concluded with some following questions. *(See attachment files for more information 01_userdata.xls, 02_brukertest.pdf and*

03_samtykkeerklæring.1.pdf, 04_samtykkeerklæring.2.pdf, 05_samtykkeerklæring.3.pdf, 06_samtykkeerklæring.4.pdf, 07_samtykkeerklæring.5.pdf)

6.3 Results

- All the users were familiar with Munch but only two attends' museums on a regular basis. They found the design simple and easy to understand.
- We were told to explain the different exhibitions better as it was unclear for the users to understand their choices.
- The tasks would be easier to understand with an explanation before they begun.
- Some icons were messy and did not have a clear meaning to them, especially the camera and map function.
- Our prototype only had one task they could test, which did not satisfy them and was not interesting enough. They thought the whole app only would have camera tasks but was explained that there would be more variety. The prototype in Figma did not allow the user to perform the task properly, as it would be in android studio with added functionality.
- They wanted a competition among other users and map visibility to see where other players were on the map/ how far they had progressed in tasks.
- One of the users said an app like this would not attract them to the museum, rather make the experience more enjoyable while they are already visiting.

6.4 Improvements

After the test, we agreed that there were some changes we wanted to include in our project or add in future development. Note: We did **NOT** make these changes after the user-test, but presented the data/solutions with the project owner during the first week (first presentation during the design sprint, Friday 26.04.2023.)

- Create a page that explains the exhibition after you have chosen it. Another alternative is to create something like a hover function after pressing the exhibition.
- Have a page that explains the tasks better after choosing an exhibition.
- Find better icons and a good placement for them.
- Include a variety of tasks as originally intended and create more tasks in the prototype on future projects.
- Find a way to implement multiplayer or parties of friends to compete against each other (or add this as an idea in the future for more development).
- Create a more interesting story or tasks that would attract new people, as they currently are more targeted to their current visitors and making it enjoyable for them.

6.5 Reflection

All the users gave a lot of the same feedback in general, which made us very satisfied with who we chose for this test. Some had participated in user tests before or had experience with IT and development, and those gave us more detailed feedback than the rest of the users. We are very satisfied with how we structured and performed the test, as it gave us time to rewind, analyze and noticed that our questions/tasks made the process smooth for all users. This made it very clear to us that we planned our test very well in order to give us the information we asked for. If we had more time, we would be more happy to test people in a more professional manner with all candidates available.

7 Further development

7.1 What would we further develop

From the beginning of the design sprint, we've had many ideas that have been discarded for different reasons. Some would just not realistically be achievable, while others were not prioritized based on time and functionality. If we were to further develop our app, there are a few functionalities we would want to implement.

First of all, we would complete the different tours by adding new tasks and games, since the app as of now only has one fully developed tour.

Regarding the technology we've based our app on, we would like to further develop the app to be responsive in different operating systems. By supporting IOS products, we would attract a much larger group of users. Our goal is for the app to be available for everyone, regardless of factors like operating system and language barriers. With that same goal in mind, we would like to implement assistive technologies such as for example a text to speech function.

A big part of the concept from the start was that we wanted the app to be a multiplayer game. In the user test we conducted, quite a few of the users expressed that they would want the app to offer either a multiplayer game or competition- rather than playing alone. That way the players could view each other's score and progress, making it more competitive. We would also want to make it possible for the user to share their results on social media, in an encouraging way. This would help promote the game and museum- bringing in more visitors.

However, the multiplayer functionality was not prioritized when we considered the time and resources it would require. We imagine that offering to play as a group would make the experience more personalized and social.

Lastly, we would further develop the existing code and functionality, making it more secure and stable.

7.2 Security reflections

To identify and assess the risks for the application we are making, we used OWASP Top 10 Mobile Risks. We have taken an evaluation of the most important elements within security for application development. When considering the elements we have used NSMs basic principles and GDPR.

NSM

We have also used NSMs basic principles. (Nasjonal sikkerhetsmyndighet, 2020) Some of the principles are similar to the information found in OWASP Top 10 Mobile Risks. Such as finding risks, and what can happen and how likely it is to occur. Lastly it is important to implement measures, such as the Principle of least privilege.

When assessing and making security decisions, we had to look at the requirements for projects within or owned by the municipality of Oslo. Because it is state owned, the rules are regulated by Digitaliseringsdirektoratets internkontrollveileder.

GDPR

When making the application and storing data, we also had to take GDPR into consideration. GDPR is a set of regulations for companies and organizations. The regulations consider how long data is kept and if it is necessary to update. It also includes how much data that can be collected. It is also important that the user is informed about which data is collected.

OWASP TOP 10

One of the risks in Owasp Mobile top 10 that we identified is Improper Platform Usage. (OWASP, 2016a) This risk includes Android intents and happens when an API call can serve as an attack vector. The impact of this vulnerability is severe, and will have a huge impact because the attacker can exploit it through the mobile device. To exploit this vulnerability, a hacker can use Cross-site scripting (XSS).

To secure the information about the users, we chose to create a database. This is because adding passwords to the code will not give secure storage. Only encrypting the data will not be the best solution, because a potential hacker could steal the data and encryption key.

The hacker would then be able to encrypt all the data. Storing the passwords in a database, then hash, salt and encrypt them is the most secure solution. The application we currently have is not built by the definition of development guidelines for security that Android has to applications on their platform, which would be the ideal solution.

Another risk found is under the category: Insecure Data Storage from OWASP Top 10. (OWASP, 2016b) In this particular case, the filesystem is easily accessible. It is not taken into consideration that a user or hacker using malware will inspect data storage. To fix this vulnerability the recommendation will be to use URL caching on both request and response side and using application backgrounding.

Insecure communication is also a risk in the MUNCH application. The application is running on HTTP, and not HTTPS. This means that the data is not encrypted, and if an attacker uses a proxy or something familiar, they will be able to read the HTTP get-, and HTTP post requests that are being sent between client and server. This risk can have a huge impact, because passwords will possibly be accessible in plaintext. Using SSL/TLS will be an alternative to improve security by encrypting the communication between client and server.

To secure authentication, the best policy is using strong passwords. By implementing a rule for what a password should consist of, we secure the users passwords from insecure authentication. Having a strong password ensures that a hacker has to use a lot more time and resources to carry out a brute force attack. A weak password policy will create insecure authentication.

Because exposed API calls can be an attack vector, insufficient cryptography is a vulnerability in the application that has been identified through OWASP top 10. (OWASP, 2016c) If this vulnerability is exploited, it can result in unauthorized retrieval of sensitive information. It is a common vulnerability in most mobile apps that use encryption. To prevent this, it is important to avoid storing data on a mobile device as much as possible. It is also important to establish cryptographic standards and follow NIST guidelines.

Insecure authorization is when a hacker passes the authentication control, and login as a legitimate user. When they have user-rights they can sometimes be able to force-browse to a vulnerable endpoint. (OWASP, 2016d) By doing this they can execute administrative functionality. To avoid these kinds of attacks, it is important to verify roles and permissions to authenticate user information that is contained in backend systems.

The code quality in the application we have made is poor in security perspective, which creates a vulnerability. The attackers often use tools to identify these vulnerabilities, such as performing fuzzing. These types of vulnerabilities are easy to exploit. Exploiting the vulnerability can result in buffer overflow. To prevent these kinds of vulnerabilities the recommendation is to have consistent coding patterns, validating the lengths of incoming buffer data and automation tools.

Code tampering is also an often seen vulnerability. This is because the mobile code runs within an environment that is controlled by another organization than the one building it. Applications with games are more vulnerable to these kinds of attacks because of the consequences. In Android Root Detection will usually the application that has been modified execute in a rooted environment or Jailbroken. It is therefore a good idea to try detecting these types of environments. (OWASP, 2016e)

All mobile code has a risk of reverse engineering. It is easy to detect ways to use reverse engineering for a hacker. They can download the application into a contained environment, and then use tools like that are binary inspection tools. This vulnerability will possibly reveal information about backend servers and perform attacks that will affect backend systems.

Extraneous functionality is also a risk when it comes to mobile applications. (OWASP, 2016f) Just like reverse engineering, the attacker downloads the application and examines the application in their own environment. To prevent these kinds of vulnerabilities we can examine API endpoints that are accessed by the mobile app.

In the code, it is a backdoor to the login feature. The user will be logged in as the last user, and will therefore not necessarily be a real user. This can be used by an attacker, because if the last person signed in is an admin user, it can possibly be a major security issue.

It is important to limit user rights. We will use the principle of least privilege (POLP). The purpose is to give the user minimum levels of access. In cybersecurity this practice is considered the best. To secure the application it is important that the requirements compliance with operational and end-user needs. (CYBERARK, 2023)

When implementing security measures we would recommend using the NIST framework. This is to follow the best practice and improve risk management. The framework consists of standards and guidelines.

If the product were to be released, we would have to carry out a penetration test. The test will show vulnerabilities that make the application insecure.

8 Result evaluation

8.1 What have we learned

Throughout this project we have gained a new understanding of how to work in an agile environment. It has been challenging, but also fun and educational. As we have gotten to know each other, we've learned how to work together as a group on a larger project.

Even though we have been quite separated when it comes to the tasks, we have all shared our knowledge and learned from each other. The non-programmers have gained a larger overall understanding of the process of building an app from start to finish. We've also made space for the group members that wanted to challenge themselves by learning new technologies and programming languages along the way. Through the process we've seen the importance of evaluating and fixing our mistakes as soon as they occur.

Another learning experience has been to listen to the product owner's feedback, even though we may not always agree with their recommendations. At one point we for example

reached out to the product owner and requested a clearer explanation regarding the feedback we had previously gotten. By making an effort to understand their point of view, we actually ended up agreeing with their recommendations after all. This experience has taught us to always be receiving and open for feedback- and not discard ideas we don't immediately agree with.

As mentioned in the project process report, we've also learned the importance of good planning, structure, and communication.

8.2 What are we satisfied with

The most loved part of our product is the thought and effort put into the overall concept. The concept is well thought through, with a clear plan of execution. All from the fact that the plot is based on a true story, to the task that can *actually* be solved at the museum right now. The design choices are based on MUNCH's own documentation and represent them well. We think the design comes off as consistently and intentional. The effort we have put into these choices has made it easier for us to show our vision and concept.

We believe our product answers the case we were given, by challenging the user to think, engage, and interpret the museum's art and physical space. The app's purpose is not to draw the user's attention away from the art, but engage them to use the physical space.

Another part of the process we are very satisfied with is the way we've adapted to the product owner's feedback. We've listened to the feedback we were given and made the changes we felt were necessary to meet their wishes. This way we have stuck to the product owner's wishes when it comes to design, functions, and elements to keep or discard. We believe we have developed an app that reflects the group's knowledge.

8.3 What would we do differently

Next time we would first of all make sure to put more effort into planning. We have learned the hard way that we should plan tasks thoroughly before delving into the programming. When working with different technologies it's also been challenging to plan the order of the

task. It has for example been difficult for the programmers to stay updated on design changes in Figma. This has on some occasions led to code that were discarded or had to be modified. From this we've learned that we need to plan design choices more thoroughly in advance and stick to them/make sure that everyone is kept up to speed on the changes.

We would also make sure to make the code into a universal solution from the start, as this had to be modified at the end.

9 Sources

- Alberge, D. (2023, 01 15). *theguardian.com*. Retrieved from How Munch's friend hid a masterpiece in a Norwegian barn to foil the nazis:
<https://www.theguardian.com/artanddesign/2023/jan/15/how-edvard-munchs-friend-hid-a-masterpiece-to-save-it-from-nazis-dance-on-the-beach>
- CYBERARK. (2023, 06 08). *cyberark.com*. Retrieved from Principle of Least Privilege:
<https://www.cyberark.com/what-is/least-privilege/>
- Nasjonal sikkerhetsmyndighet. (2020, 08 26). *nsm.no*. Retrieved from Grunnprinsipper for IKT-sikkerhet: <https://nsm.no/fagomrader/digital-sikkerhet/rad-og-anbefalinger-innenfor-digital-sikkerhet/grunnprinsipper-ikt>
- Icons8 Team, (2023). *Material Design Icons*
<https://www.figma.com/community/plugin/740272380439725040>
- OWASP. (2016b). *owasp.org*. Retrieved from M2: Insecure Data Storage:
<https://owasp.org/www-project-mobile-top-10/2016-risks/m2-insecure-data-storage>
- OWASP. (2016c). *owasp.org*. Retrieved from M5: insufficient Cryptography:
<https://owasp.org/www-project-mobile-top-10/2016-risks/m5-insufficient-cryptography>
- OWASP. (2016d). *owasp.org*. Retrieved from M6: Insecure Authorization:
<https://owasp.org/www-project-mobile-top-10/2016-risks/m6-insecure-authorization>
- OWASP. (2016e). *owasp.org*. Retrieved from M8: Code Tampering:
<https://owasp.org/www-project-mobile-top-10/2016-risks/m8-code-tampering>
- OWASP. (2016f). *owasp.org*. Retrieved from M10: Extraneous Functionality:
<https://owasp.org/www-project-mobile-top-10/2016-risks/m10-extraneous-functionality>

OWASP. (2016a). *Owasp.org*. Retrieved from M1: Improper Platform Usage:
<https://owasp.org/www-project-mobile-top-10/2016-risks/m1-improper-platform-usage>

10 Attachments

00_sqlseed.sql

01_userdata.xls

02_brukertest.pdf

03_samtykkeerklæring.1.pdf

04_samtykkeerklæring.2.pdf

05_samtykkeerklæring.3.pdf

06_samtykkeerklæring.4.pdf

07_samtykkeerklæring.5.pdf

7. Sources

Login system

<https://www.youtube.com/watch?v=ayKMfVt2Sg4>

Project Result